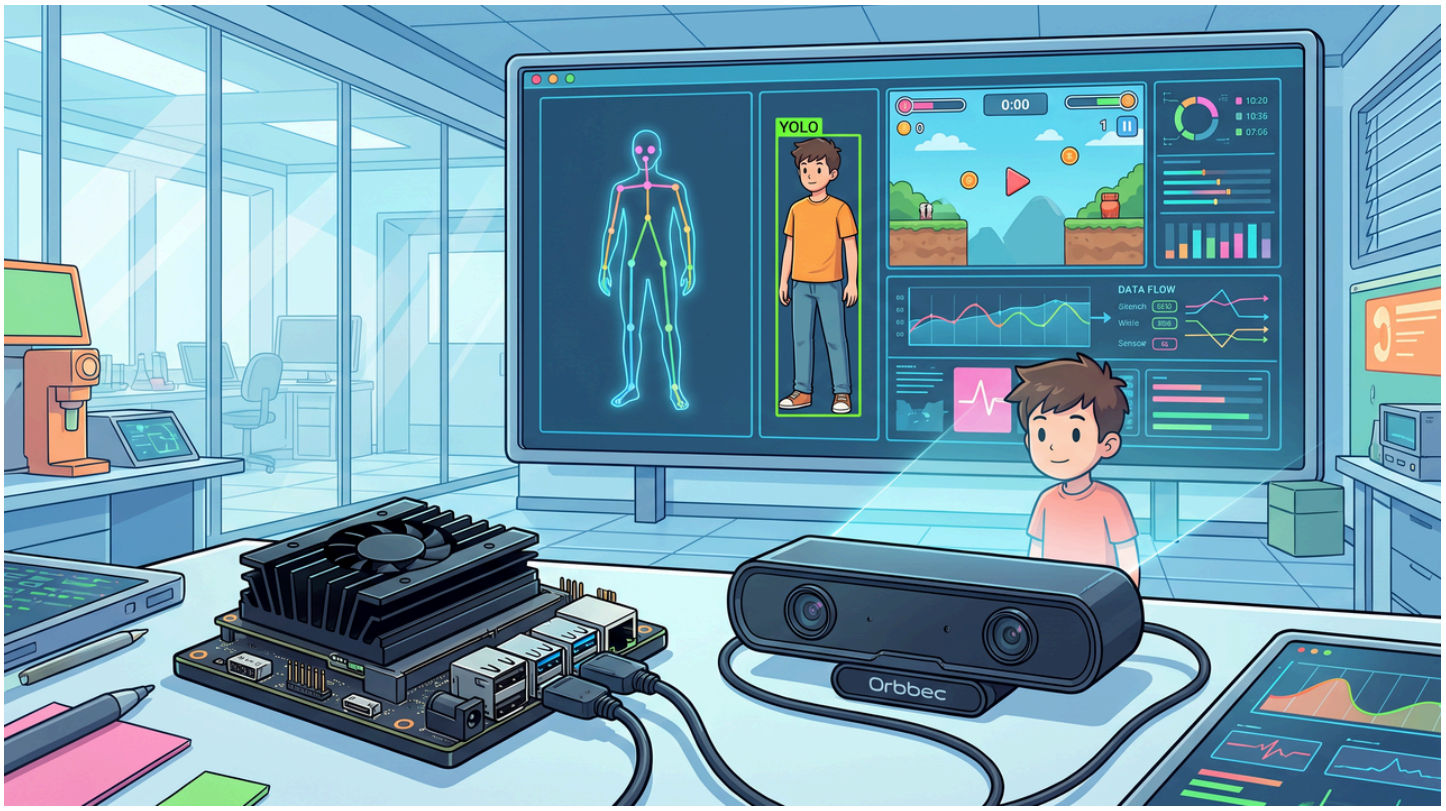
🚀 把「公共场所安全监控与互动小游戏」跑上 Jetson

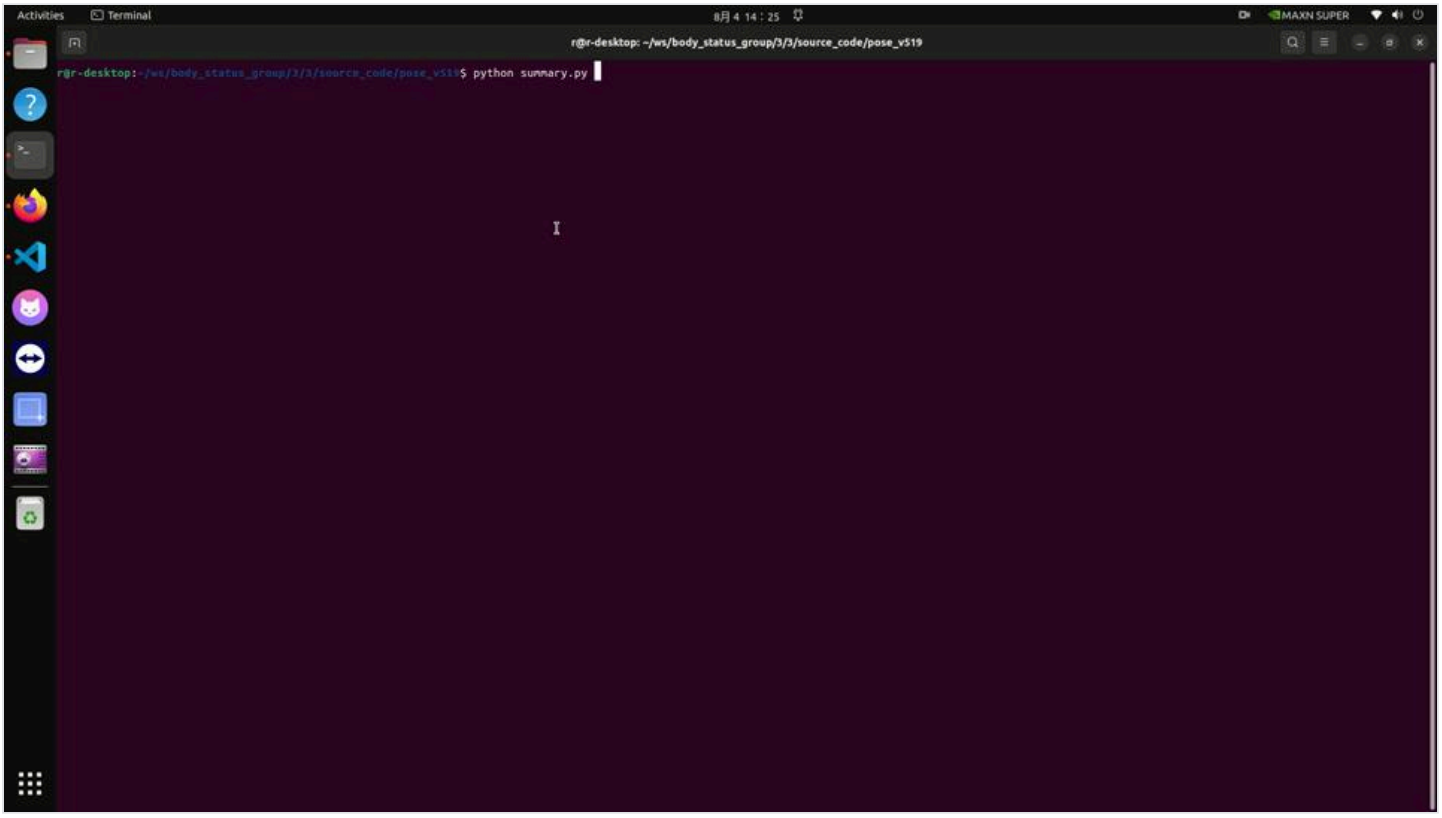


项目	说明
适用项目目录	`pose_v519`
适用入口脚本	`summary.py`
目标设备	NVIDIA Jetson Orin / Xavier 系列优先，Nano 系列需额外评估依赖兼容性
项目改编自	深圳大学创新短课项目：人体识别和姿态检测（李鹏飞，刘思昊，范倚彰，罗杜）
文档日期	2026-08-04

1. 项目一句话说明

本项目将 Orbbec 深度相机、YOLO ONNX 模型、MediaPipe Pose Landmarker、OpenCV 窗口界面和 pygame 声音模块组合在一起，实现两个功能：

功能	说明
安全监控	多人姿态检测、活体判断、摔倒报警
互动游戏	基于人体动作识别的动作模仿小游戏



在 Jetson 系列设备上复现时，难点通常不在 Python 语法，而在 **ARM64 软件包兼容、相机权限、OpenCV GUI、中文字体、音频声道、ONNX Runtime / TensorRT 推理后端** 这些工程细节上。

---

2. 推荐硬件与系统版本

2.1 推荐设备

推荐等级	设备
⭐ 优先推荐	Jetson Orin Nano / Orin NX / AGX Orin
⚠️ 可尝试（需更多适配）	Jetson Xavier NX / AGX Xavier
❌ 不优先推荐	Jetson Nano 4GB / 2GB（系统较旧、Python 版本低、ARM64 wheel 支持有限）

2.2 JetPack 选择建议

建议使用 NVIDIA 官方 JetPack / Jetson Linux 发行版，**不建议自行拼装内核、CUDA、TensorRT**。

参考资料：

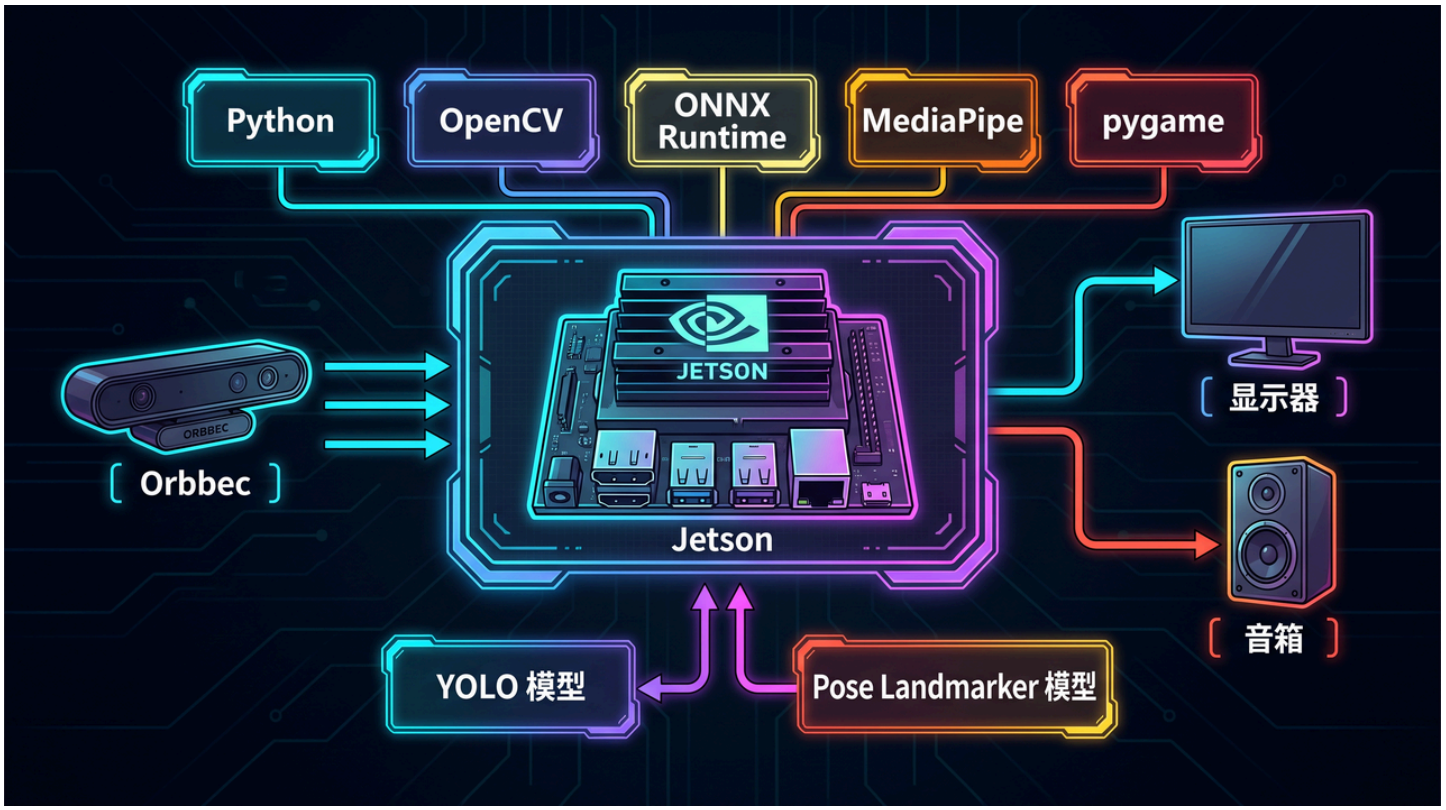
- NVIDIA JetPack 官方文档
- Jetson Linux 开发者指南
- TensorRT 官方文档

经验建议：

设备系列	建议 JetPack 版本
Orin 系列	JetPack 6.x
Xavier 系列	JetPack 5.x

> 💡 如果学校/实验室/项目组已有统一镜像，以统一镜像为准，但要记录 JetPack、Python、OpenCV、CUDA、TensorRT 版本。

3. 系统准备



3.1 更新系统基础包

代码块

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y \
  git cmake build-essential pkg-config \
  python3 python3-pip python3-venv python3-dev \
  libgl1 libglib2.0-0 libgtk-3-0 libgtk-3-dev \
  libjpeg-dev libpng-dev libtiff-dev \
  libsdl2-2.0-0 pulseaudio-utils \
  fonts-noto-cjk fonts-wqy-microhei
```

各依赖说明：

依赖包	作用
`libgtk-3-0` / `libgtk-3-dev`	保证 OpenCV HighGUI 可以创建窗口
`fonts-noto-cjk` / `fonts-wqy-microhei`	保证中文标题、按钮、HUD 可以被 PIL 正常绘制
`libsdl2-2.0-0` / `pulseaudio-utils`	pygame 声音模块常用依赖

3.2 开启高性能模式

代码块

```
sudo nvpmodel -q
sudo jetson_clocks
```

> ⚠️ 不同 Jetson 型号的 `nvpmodel` 模式编号不同，**不要盲目复制** `sudo nvpmodel -m 0`。先用 `nvpmodel -q` 查看当前设备支持的功耗模式，再按实际情况切换。

3.3 可选：增加 swap

如果设备内存较小，安装 Python 依赖或编译包时容易 OOM，可以增加 swap：

代码块

```
sudo fallocate -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
free -h
```

> 长期使用时再写入 `/etc/fstab`，临时复现阶段可以先不持久化。

---

4. 获取项目代码与模型文件

建议目录结构：

代码块

```
pose_v519/
├── summary.py
├── utils.py
├── 鸽子舞.mp3
├── coco.names
├── models/
│   ├── yolo11n.onnx
│   ├── yolo26n.onnx
│   └── pose_landmarker_lite.task
└── game_frontend/
```

确认关键文件存在：

代码块

```
cd ~/ws/body_status_group/3/3/source_code/pose_v519
ls -lh summary.py utils.py coco.names 鸽子舞.mp3
ls -lh models/yolo11n.onnx models/yolo26n.onnx models/pose_landmarker_lite.task
```

> ⚠️ 如果模型文件缺失，程序会在加载 YOLO 或 MediaPipe 时失败。

---

5. Python 环境创建

5.1 推荐方式：虚拟环境

Jetson 上经常需要复用系统自带 OpenCV，因此推荐创建 **带系统包可见性** 的虚拟环境：

代码块

```
cd ~/ws/body_status_group/3/3/source_code/pose_v519
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
```

5.2 不建议直接全量执行原始 requirements.txt

项目里的 `requirements.txt` 面向普通 PC / Windows 环境的痕迹较多：

有问题的依赖	原因
`onnxruntime-directml`	DirectML 主要用于 Windows，不适合 Jetson
`PySide6==...`	只运行 `summary.py` 时不是必要依赖
`opencv-python` + `opencv-contrib-python`	同时安装容易互相覆盖

建议按功能逐项安装，而不是一口气 `pip install -r requirements.txt`。

---

6. 依赖安装建议

6.1 OpenCV

优先检查系统是否已有 OpenCV：

代码块

```
import cv2
print("OpenCV:", cv2.__version__)
info = cv2.getBuildInformation()
if "GUI:" in info:
    print(info.split("GUI:")[1].splitlines()[0])
else:
    print("GUI info not found")
```

验证要求：

- ✅ 能 `import cv2`
- ✅ GUI 后端 **不是** `NONE`
- ❌ 不要安装 `opencv-python-headless`

如果 `cv2` 不存在，可以先尝试：

代码块

```
sudo apt install -y python3-opencv
```

如果必须使用 pip 版 OpenCV，请注意 ARM64 wheel 与 JetPack / Python 版本兼容性。安装后务必确认 GUI 可用。

6.2 基础 Python 包

代码块

```
pip install numpy pillow pygame
```

6.3 ONNX Runtime

快速复现阶段可以先使用 CPU Provider：

代码块

```
pip install onnxruntime
```

验证：

代码块

```
import onnxruntime as ort
print(ort.__version__)
print(ort.get_available_providers())
```

正常输出示例：

代码块

```
['CPUExecutionProvider']
```

> 说明可以运行，但推理速度依赖 CPU。

Jetson 上不要安装：

代码块

```
# ❌ 错误示范 —— DirectML 不是 Jetson 的 GPU 加速路径
pip install onnxruntime-directml
```

Jetson 上的高性能路线通常是 **TensorRT** 或 **ONNX Runtime TensorRT Execution Provider**，但这要求版本严格匹配 CUDA、TensorRT、JetPack 和 Python。**建议先跑通 CPU 版本，再做加速优化。**

6.4 MediaPipe

尝试安装：

代码块

```
pip install mediapipe
```

如果提示没有可用 wheel，说明当前 Python / ARM64 / JetPack 组合不被 PyPI wheel 覆盖。此时有三种处理方式：

1. 更换到项目组已验证的 JetPack + Python 版本
2. 使用已验证的 ARM64 预编译 wheel
3. 从源码构建 MediaPipe

> ⚠️ 不要在没有记录版本的情况下反复混装，否则后面排查会非常困难。

6.5 Orbbec SDK

项目依赖 `pyorbbecsdk`，可以先尝试：

代码块

```
pip install pyorbbecsdk2
```

参考资料：Orbbec SDK Python Wrapper (GitHub)

安装后验证：

代码块

```
from pyorbbecsdk import *
print("pyorbbecsdk import ok")
```

> 如果相机无法打开，优先检查 USB 权限和 udev 规则。Orbbec 官方仓库通常提供环境配置脚本或 udev 规则，请按对应 SDK 版本执行。

---

7. Orbbec 相机连接检查



7.1 USB 连接

代码块

```
lsusb
```

确认能看到 Orbbec 设备。

7.2 权限问题

如果普通用户打不开相机，但 `sudo python summary.py` 可以打开，多半是 udev 权限未配置。

处理方式：

代码块

```
# 1. 查看当前用户组
groups

# 2. 将用户加入 video 和 plugdev 组
sudo usermod -aG video,plugdev $USER

# 3. 注销重新登录后生效
```

如果 SDK 提供 udev rules，请复制到 `/etc/udev/rules.d/` 并执行：

代码块

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

7.3 USB 带宽

深度相机建议 **直接接 Jetson 主机 USB 3.0 口**，不建议接无源 HUB。若频繁丢帧、启动失败或深度帧为空，优先换线、换口、减少同总线设备。

---

8. 运行前检查清单

进入项目目录并激活环境：

```
代码块
1  cd ~/ws/body_status_group/3/3/source_code/pose_v519
2  source .venv/bin/activate
```

逐项检查：

① 检查 Python 版本

```
代码块
1  python --version
```

② 检查模型文件

```
代码块
1  test -f models/yolo11n.onnx && echo "✅ yolo11n ok"
2  test -f models/yolo26n.onnx && echo "✅ yolo26n ok"
3  test -f models/pose_landmarker_lite.task && echo "✅ pose landmarker ok"
```

③ 检查 OpenCV GUI

代码块

```
import cv2
import numpy as np

win = "opencv_gui_test"
cv2.namedWindow(win, cv2.WINDOW_NORMAL)
cv2.imshow(win, np.zeros((240, 320, 3), dtype=np.uint8))
cv2.waitKey(500)
cv2.destroyAllWindows()
print("opencv gui ok")
```

> ⚠️ 如果这里失败，**不要继续跑主程序**，先处理桌面环境或 OpenCV GUI。

④ 检查中文字体

代码块

```
fc-list | grep -Ei "NotoSansCJK|WenQuanYi|SourceHan|SimHei|Microsoft YaHei" |
head
```

⑤ 检查音频

代码块

```
import pygame
pygame.mixer.init()
print("pygame mixer:", pygame.mixer.get_init())
```

---

9. 正式运行

代码块

```
cd ~/ws/body_status_group/3/3/source_code/pose_v519
source .venv/bin/activate
python summary.py
```

正常启动时应看到：

代码块

```
[信息] 背景音乐加载成功: 鸽子舞.mp3
公共场所安全监控与互动小游戏 — 启动器
点击按钮选择功能，按 ESC 退出
```

进入互动游戏后，正常流程类似：

代码块

```
[信息] ONNX Runtime 可用提供程序: [...]
[信息] YOLOv11n ONNX 模型加载成功
[信息] MediaPipe Pose Landmarker 初始化成功
[信息] 相机已启动
[信息] 相机内参: fx=..., fy=..., cx=..., cy=...
动作模仿游戏已启动
```

---

10. 已遇到并修复过的问题

10.1 OpenCV 报 `NULL window handler`

现象：

```
代码块
1   cv2.error: NULL window handler in function 'cvSetMouseCallback'
```

原因：

- `cv2.setMouseCallback()` 执行时，OpenCV 窗口句柄还没有真正创建完成
- Linux Qt / GTK HighGUI 对中文窗口名也可能不稳定

修复策略：

- OpenCV 内部窗口名改为 ASCII：`launcher`、`monitor`、`game`
- 创建窗口后先 `imshow` 一帧空画面，再 `waitKey(1)`，最后绑定鼠标回调

避坑建议：

- OpenCV 窗口名尽量使用英文
- 绑定鼠标回调前确认窗口已经初始化
- SSH 无图形环境下不要直接运行 GUI 程序

10.2 PIL 报 `Only supported for TrueType fonts`

现象：

```
代码块
1   ValueError: Only supported for TrueType fonts
```

原因：

- 系统没有加载到 TrueType 中文字体
- PIL 回退到了默认位图字体，而 `textbbox()` 不支持该字体

修复策略：

- 自动查找 Linux / Windows 常见中文字体
- 文本测量改为兼容函数，不再直接依赖 `draw.textbbox()`

Jetson 上建议安装：

```
代码块
1   sudo apt install -y fonts-noto-cjk fonts-wqy-microhei
```

10.3 pygame 报 `Array must be 2-dimensional for stereo mixer`

现象：

```
代码块
```

```
ValueError: Array must be 2-dimensional for stereo mixer
```

原因：

- pygame mixer 初始化为双声道
- 代码生成的提示音数组是一维单声道

修复策略：

- 根据 `pygame.mixer.get_init()` 自动判断声道数
- 如果是双声道，自动把音频数组扩展为二维
- 提示音失败时仅打印警告，不影响主流程

10.4 ONNX Runtime GPU discovery warning

现象：

代码块

```
GPU device discovery failed
```

说明：

- 这通常 **不是致命错误**
- 如果后续能加载模型并进入界面，说明程序仍在 CPU Provider 上运行

Jetson 注意：

- ❌ 不要安装 `onnxruntime-directml`（DirectML 是 Windows 路线）
- ✅ Jetson 加速建议后续走 TensorRT 或 ONNX Runtime TensorRT EP

10.5 MediaPipe warning

现象：

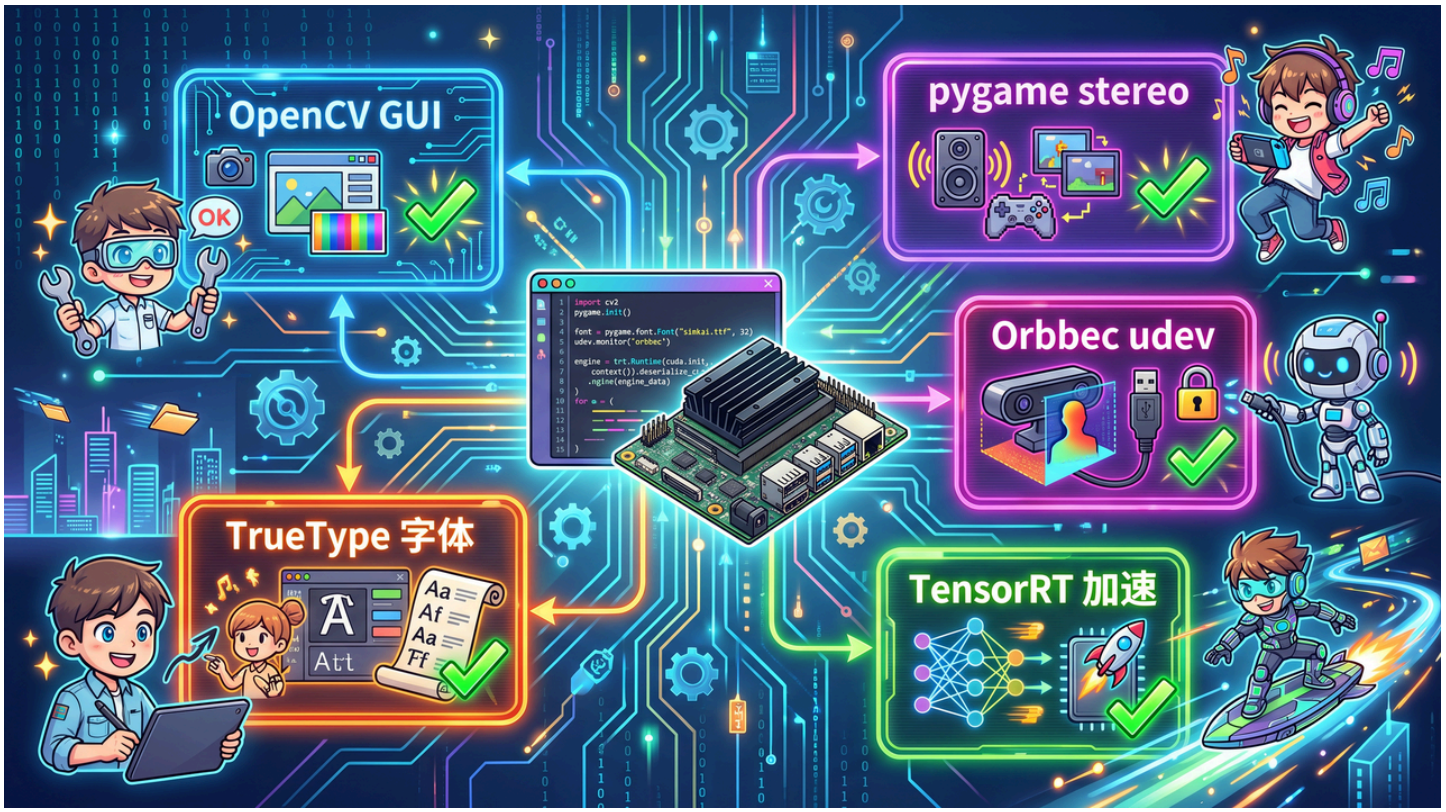
```
代码块
1  Feedback manager requires a model with a single signature inference
2  landmark_projection_calculator.cc: Using NORM_RECT without IMAGE_DIMENSIONS...
```

说明：

- 这些通常是 MediaPipe 内部 warning
- 如果人体关键点能正常输出，可以 **先忽略**

---

11. Jetson 复现时最常见的坑



坑 1️⃣：在 SSH 无桌面环境运行 OpenCV GUI

错误表现：

- OpenCV 窗口创建失败
- `setMouseCallback` 找不到窗口
- 程序没有画面

处理方式：

- 在 Jetson **本机桌面终端**运行
- 或正确配置 X11 / Wayland 转发
- 如果通过 SSH 调用本机桌面，可以尝试：

代码块

```
export DISPLAY=:0
export XAUTHORITY=/home/$USER/.Xauthority
python summary.py
```

坑 2️⃣：安装了 `opencv-python-headless`

错误表现：

- `cv2.imshow()` 不可用
- HighGUI 报错

处理方式：

```
代码块
1   pip uninstall -y opencv-python-headless
```

然后使用系统 OpenCV 或带 GUI 的 OpenCV。

坑 3️⃣：同时安装多个 OpenCV 包

不要同时混装：

```
代码块
1   opencv-python
2   opencv-contrib-python
3   opencv-python-headless
4   python3-opencv
```

至少要明确当前 Python 实际 import 的是哪一个：

代码块

```
import cv2
print(cv2.__file__)
print(cv2.__version__)
```

坑 4️⃣：照搬 PC 端 requirements

Jetson 是 ARM64，不是 x86_64。很多 pip wheel 是否可用取决于：

影响因素
JetPack 版本
Ubuntu 版本
Python 版本
CUDA / TensorRT 版本
包是否发布了 Linux aarch64 wheel

> 建议 **逐项安装、逐项验证**。

坑 5️⃣：相机权限没有配置

表现：

- `lsusb` 能看到相机
- Python 程序打不开相机
- sudo 能打开，普通用户打不开

> 优先处理 udev rules 和用户组权限。

坑 6️⃣：把 CPU 跑通和 GPU 加速混为一谈

建议分两阶段：

1. **先用 CPU Provider 跑通完整业务流程**
2. **再做 TensorRT / CUDA / ONNX Runtime EP 加速**

> 不要一开始就追求 GPU Provider，否则问题会混在一起，很难判断是依赖问题、模型问题还是相机问题。

---

12. 建议的最小可复现顺序

> 严肃建议按下面顺序来，**不要跳步**。

步骤	操作	验证标准
1	系统启动到 Jetson 本机桌面	能看到桌面环境

2	`lsusb` 确认 Orbbec 相机存在	能看到 Orbbec 设备
3	Python 虚拟环境创建成功	`which python` 指向 `.venv`
4	`import cv2` 成功	无报错
5	OpenCV GUI 小窗口测试成功	弹出测试窗口
6	中文字体安装并能被 `fc-list` 查到	有 Noto / WenQuanYi 输出
7	`import pygame` 和 `pygame.mixer.init()` 成功	mixer 初始化返回非空
8	`import onnxruntime` 成功	能打印版本号和 providers
9	`import pyorbbecsdk` 成功	无报错
10	`import mediapipe` 成功	无报错
11	确认三个模型文件存在	`test -f` 全部通过
12	运行 `python summary.py`	启动器窗口出现
13	先进入启动器，不点功能	窗口稳定不崩溃
14	再进入互动游戏	相机、模型、MediaPipe 初始化成功
15	最后测试安全监控功能	全部功能可进入、可退出

---

13. 性能优化建议

13.1 降低推理压力

项目中已有跳帧策略：

代码块

```
INFERENCE_SKIP = 8
```

如果 Jetson 设备帧率不够，可以适当增大跳帧值：

代码块

```
INFERENCE_SKIP = 10  # 代价是动作响应会更慢
```

13.2 降低相机分辨率

当前日志里彩色配置可能是 `1280x720`。如果 CPU 推理压力较大，可以优先考虑降低彩色分辨率或减少显示处理量。

13.3 TensorRT 加速路线

生产部署建议把 YOLO ONNX 转成 TensorRT engine：

代码块

```
/usr/src/tensorrt/bin/trtexec \
  --onnx=models/yolo11n.onnx \
  --saveEngine=models/yolo11n_fp16.engine \
  --fp16
```

注意事项：

- 这一步只是生成 engine 文件
- 当前 `summary.py` 使用 ONNX Runtime 加载 `.onnx`，如果要直接加载 `.engine`，**需要改推理代码**

- 如果使用 ONNX Runtime TensorRT Execution Provider，也要确保 ONNX Runtime、CUDA、TensorRT、JetPack 版本匹配

---

14. 出问题时如何记录日志

建议每次复现都保存完整日志：

代码块

```
python summary.py 2>&1 | tee run_$(date +%Y%m%d_%H%M%S).log
```

日志里重点看：

关注项	说明
OpenCV 版本和 GUI 后端	确认不是 headless
ONNX Runtime providers	确认可用推理后端
MediaPipe 是否初始化成功	关键依赖
Orbbec 相机彩色流/深度流配置	相机是否正常出帧
相机内参是否输出	确认 intrinsics 读取成功
是否出现 Python traceback	定位报错位置

---

15. 提交环境信息模板

复现失败时，请按以下模板填写环境信息：

代码块

```
【设备信息】
设备型号：
JetPack 版本：
Ubuntu 版本：
Python 版本：

【关键依赖版本】
OpenCV 版本：
OpenCV GUI 后端：
ONNX Runtime 版本：
ONNX Runtime Providers：
MediaPipe 版本：
pyorbbecsdk 版本：

【运行环境】
是否本机桌面运行：
是否能 lsusb 看到 Orbbec：
是否安装中文字体：
运行入口：

【报错信息】
完整报错日志（粘贴或附 log 文件）：
```

---

16. 最终验收标准

一次 Jetson 复现可以认为 **成功**，需要满足以下全部条件：

序号	验收项	状态
1	启动器窗口正常显示	☐
2	鼠标点击按钮不触发 OpenCV 窗口句柄错误	☐

3	中文标题、按钮、HUD 正常显示或不导致程序退出	☐
4	pygame 背景音乐或提示音不刷 traceback	☐
5	Orbbec 相机成功启动	☐
6	输出彩色流、深度流配置	☐
7	YOLO ONNX 模型加载成功	☐
8	MediaPipe Pose Landmarker 初始化成功	☐
9	互动游戏可进入、可退出	☐
10	安全监控可进入、可退出	☐
11	程序最后能安全释放资源并退出	☐

---

17. 简短结论

Jetson 复现这类「相机 + AI 推理 + GUI + 声音」项目时，请始终记住一个原则：

> 🎯 **先跑通完整链路，再优化性能；先确认系统依赖，再处理模型加速。**

本项目在调试过程中已经暴露并修复了三个典型跨平台问题：

- ✅ OpenCV 窗口句柄初始化问题
- ✅ PIL 中文字体 TrueType 兼容问题
- ✅ pygame stereo mixer 音频数组维度问题

后续如果要做正式部署，建议 **单独开一轮 Jetson TensorRT 加速适配**，把 YOLO 推理从 CPU Provider 迁移到 TensorRT 路线。这样既能保持当前业务逻辑稳定，又能逐步提升实时性能。